

INSTITUTO FEDERAL DO SUL DE MINAS GERAIS
Campus Poços de Caldas

PROJETO INTERDISCIPLINAR

Banco de Dados II & Programação Web II

Sistema de Gerenciamento de Biblioteca de Jogos

Relatório técnico do projeto web integrado ao banco de dados

Integrantes

Adrian Soares

Carlos Eduardo Gouvea

Luiz Otavio Freire

Ryan Andreoli

Otavio Trindade

Poços de Caldas - MG

1º Semestre / 2026

SUMÁRIO

1. Apresentação do projeto
2. Banco de Dados
 - 2.1 Ideia do modelo
 - 2.2 MER e DER
 - 2.3 Entidades, atributos e relacionamentos
 - 2.4 Modelo relacional
 - 2.5 Normalização e dependências funcionais
3. Implementação SQL
 - 3.1 Criação das tabelas
 - 3.2 Inserção de dados
 - 3.3 Consultas utilizadas no trabalho
 - 3.4 View de relatório
4. Sistema Web
 - 4.1 Tecnologias e arquitetura
 - 4.2 Estrutura de pastas e rotas
 - 4.3 Trechos principais do código
5. Execução, testes e fechamento

1. APRESENTAÇÃO DO PROJETO

A ideia do nosso grupo foi criar uma biblioteca de jogos simples, mas com as partes principais que um sistema real teria: cadastro dos jogos, organização por gênero, indicação da desenvolvedora, plataformas disponíveis e avaliações feitas pelos usuários.

O projeto junta as duas disciplinas porque o banco de dados não fica separado do sistema web. Primeiro foi feita a modelagem, pensando nas tabelas e nos relacionamentos. Depois, essa estrutura foi usada dentro da aplicação em Node.js, com páginas para cadastrar, listar, editar, excluir, avaliar jogos e consultar relatórios.

Resumo do que o sistema faz na prática

O usuário acessa o sistema pelo navegador, cadastra um jogo com gênero, desenvolvedora e plataforma, pode editar esse cadastro, excluir quando necessário, registrar avaliações e visualizar um relatório com média das notas e quantidade de avaliações.

Item	Descrição
Nome do projeto	Sistema de Gerenciamento de Biblioteca de Jogos
Tema escolhido	Biblioteca digital para controle de jogos e avaliações
Integrantes	Adrian Henrique Soares, Carlos Eduardo, Luiz, Ryan e Otavio
Banco de dados	MySQL
Back-end	Node.js com Express
Front-end	EJS, HTML, CSS e Bootstrap
Conexão com o banco	mysql2

O sistema foi pensado para ser direto de apresentar: a tela inicial leva para a listagem dos jogos, o cadastro insere dados reais no banco e o relatório mostra os resultados das avaliações. Assim, dá para demonstrar tanto a parte de Banco de Dados quanto a parte de Programação Web II funcionando juntas.

2. BANCO DE DADOS

2.1 Ideia do modelo

Na modelagem, o jogo ficou como a entidade central. A partir dele, o banco se conecta com gênero, desenvolvedora, plataforma e avaliação. Essa escolha deixa o modelo mais organizado, porque cada informação fica em uma tabela própria e o jogo apenas guarda as chaves estrangeiras necessárias.

A tabela de usuários foi incluída para registrar quem faz uma avaliação. Mesmo sendo um sistema simples, isso deixa claro que uma nota não fica solta no banco: ela pertence a um usuário e também pertence a um jogo.

A relação entre jogo e plataforma recebeu uma atenção maior, porque um mesmo jogo pode sair para PC, PlayStation, Xbox e outras plataformas. Ao mesmo tempo, cada plataforma também pode ter vários jogos. Por isso, foi criada uma tabela associativa chamada jogo_plataforma.

Decisão importante da modelagem

Não colocamos as plataformas dentro da tabela jogo como um texto único, porque isso deixaria o banco desorganizado e dificultaria consultas. Com a tabela jogo_plataforma, o relacionamento N:N fica correto e o sistema consegue buscar as plataformas de cada jogo usando JOIN.

2.2 Modelo Entidade-Relacionamento (MER)

O MER apresenta as entidades principais do sistema e mostra de forma visual como elas se relacionam. Nele aparecem os relacionamentos 1:N entre usuário e avaliação, jogo e avaliação,

gênero e jogo, desenvolvedora e jogo. Também aparece o relacionamento N:N entre jogo e plataforma, resolvido pela entidade associativa JOGO_PLATAFORMA.

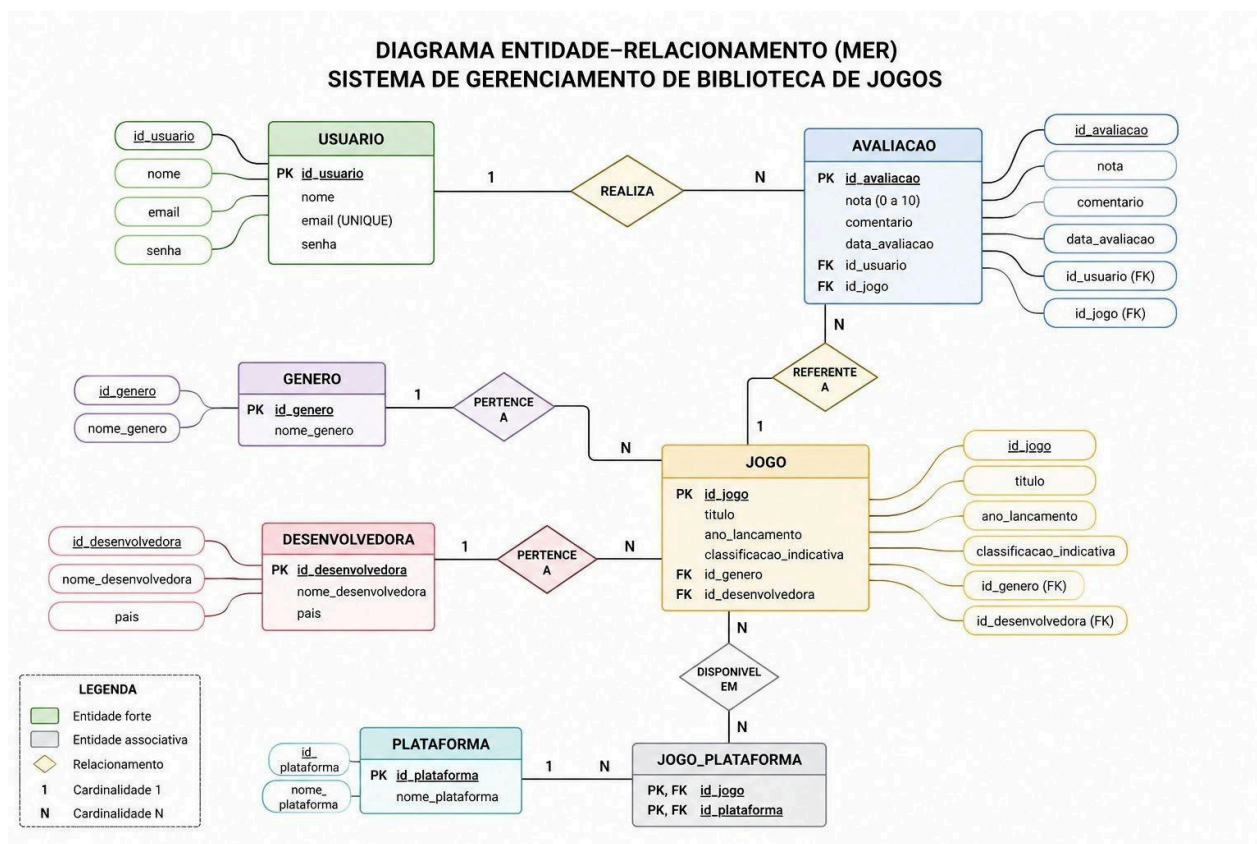


Figura 1 - Modelo Entidade-Relacionamento do sistema de Biblioteca de Jogos.

2.3 Diagrama Entidade-Relacionamento (DER)

O DER mostra a estrutura física do banco de dados no formato usado pelo MySQL Workbench. Nele ficam visíveis as tabelas, tipos de atributos, chaves primárias, chaves estrangeiras e índices. Essa parte foi usada como base para montar o script SQL final.

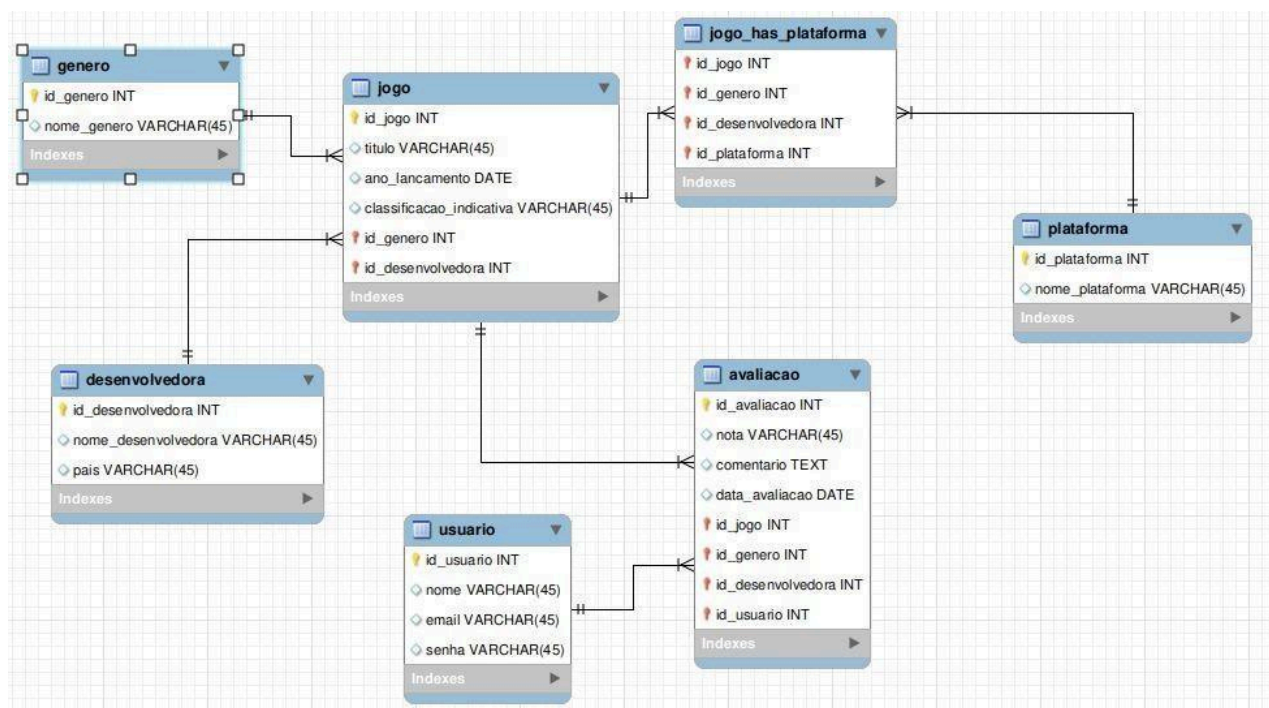


Figura 2 - DER físico gerado para a Biblioteca de Jogos.

2.4 Entidades e atributos

As entidades foram separadas de acordo com o papel de cada informação dentro do sistema. A tabela abaixo resume a função de cada uma delas.

Entidade	Função no sistema	Atributos principais
usuario	Guarda quem realiza avaliações.	id_usuario, nome, email, senha
genero	Classifica o tipo principal de cada jogo.	id_genero, nome_genero
desenvolvedora	Armazena a empresa responsável pelo jogo.	id_desenvolvedora, nome_desenvolvedora, pais
plataforma	Registra onde o jogo está disponível.	id_plataforma, nome_plataforma
jogo	Tabela central do sistema.	id_jogo, titulo, ano_lancamento, classificacao_indicativa, id_genero, id_desenvolvedora
avaliacao	Registra nota e comentário sobre um jogo.	id_avaliacao, nota, comentario, data_avaliacao, id_usuario, id_jogo
jogo_plataforma	Liga jogos e plataformas no relacionamento N:N.	id_jogo, id_plataforma

2.5 Relacionamentos e cardinalidades

- Genero 1:N Jogo - um gênero pode aparecer em vários jogos, mas cada jogo possui um gênero principal.
- Desenvolvedora 1:N Jogo - uma desenvolvedora pode ter vários jogos cadastrados, enquanto cada jogo aponta para uma desenvolvedora principal.
- Usuario 1:N Avaliacao - um usuário pode fazer várias avaliações, mas cada avaliação pertence a um único usuário.
- Jogo 1:N Avaliacao - um jogo pode receber várias avaliações, enquanto cada avaliação se refere a um único jogo.
- Jogo N:N Plataforma - um jogo pode estar em várias plataformas e uma plataforma pode possuir vários jogos. Esse relacionamento foi implementado pela tabela jogo_plataforma.

2.6 Modelo relacional

Depois do MER e do DER, o modelo foi mapeado para tabelas relacionais. A representação abaixo mostra as chaves primárias e estrangeiras usadas no banco.

Tabela	Representação relacional
usuario	usuario(id_usuario [PK], nome, email [UNIQUE], senha)
genero	genero(id_genero [PK], nome_genero [UNIQUE])
desenvolvedora	desenvolvedora(id_desenvolvedora [PK], nome_desenvolvedora, pais)
plataforma	plataforma(id_plataforma [PK], nome_plataforma [UNIQUE])
jogo	jogo(id_jogo [PK], titulo, ano_lancamento, classificacao_indicativa, id_genero [FK], id_desenvolvedora [FK])
avaliacao	avaliacao(id_avaliacao [PK], nota, comentario, data_avaliacao, id_usuario [FK], id_jogo [FK])
jogo_plataforma	jogo_plataforma(id_jogo [PK/FK], id_plataforma [PK/FK])

Com esse modelo, os dados de exemplo comprovam os relacionamentos: há jogos ligados a gêneros, desenvolvedoras e plataformas, além de avaliações ligadas a usuários e jogos.

2.7 Normalização

A normalização foi aplicada separando as informações por assunto. Isso evita que dados como nome do gênero, nome da plataforma ou desenvolvedora fiquem repetidos em vários registros da tabela jogo.

A 1FN é atendida porque os campos possuem valores simples, sem listas dentro de uma coluna. A 2FN é atendida porque os atributos dependem da chave da tabela. A 3FN também é atendida, pois não foram colocados atributos que dependem de outros atributos não-chave.

O melhor exemplo disso é a tabela jogo_plataforma. Sem ela, seria necessário escrever várias plataformas em um único campo de texto no cadastro do jogo. Com ela, o banco fica mais organizado e as consultas com JOIN ficam mais corretas.

2.8 Dependências funcionais

Tabela	Dependência funcional
usuario	id_usuario -> nome, email, senha
genero	id_genero -> nome_genero
desenvolvedora	id_desenvolvedora -> nome_desenvolvedora, pais
plataforma	id_plataforma -> nome_plataforma
jogo	id_jogo -> titulo, ano_lancamento, classificacao_indicativa, id_genero, id_desenvolvedora
avaliacao	id_avaliacao -> nota, comentario, data_avaliacao, id_usuario, id_jogo
jogo_plataforma	(id_jogo, id_plataforma) -> vínculo entre jogo e plataforma

3. IMPLEMENTAÇÃO SQL

3.1 Criação das tabelas

O script SQL cria o banco biblioteca_jogos e todas as tabelas do sistema. Foram usadas chaves primárias, chaves estrangeiras, restrições UNIQUE, CHECK e comandos DROP TABLE IF EXISTS para facilitar a execução novamente durante os testes.

```
DROP DATABASE IF EXISTS biblioteca_jogos;

CREATE DATABASE biblioteca_jogos
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

USE biblioteca_jogos;

CREATE TABLE usuario (
  id_usuario INT AUTO INCREMENT PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE,
  senha VARCHAR(100) NOT NULL
);

CREATE TABLE genero (
  id_genero INT AUTO INCREMENT PRIMARY KEY,
  nome_genero VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE desenvolvedora (
  id_desenvolvedora INT AUTO INCREMENT PRIMARY KEY,
  nome_desenvolvedora VARCHAR(100) NOT NULL,
  pais VARCHAR(50) NOT NULL
);

CREATE TABLE plataforma (
  id_plataforma INT AUTO INCREMENT PRIMARY KEY,
  nome_plataforma VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE jogo (
  id_jogo INT AUTO INCREMENT PRIMARY KEY,
  titulo VARCHAR(100) NOT NULL,
  ano_lancamento INT NOT NULL,
  classificacao_indicativa VARCHAR(10) NOT NULL,
  id_genero INT NOT NULL,
  id_desenvolvedora INT NOT NULL,

  CONSTRAINT fk_jogo_genero
    FOREIGN KEY (id_genero)
    REFERENCES genero(id_genero)
    ON DELETE RESTRICT
    ON UPDATE CASCADE,

  CONSTRAINT fk_jogo_desenvolvedora
    FOREIGN KEY (id_desenvolvedora)
    REFERENCES desenvolvedora(id_desenvolvedora)
    ON DELETE RESTRICT
    ON UPDATE CASCADE,

  CONSTRAINT chk_ano
    CHECK (ano_lancamento >= 1970)
);

CREATE TABLE avaliacao (
  id_avaliacao INT AUTO INCREMENT PRIMARY KEY,
  nota DECIMAL(3,1) NOT NULL,
  comentario TEXT,
  data_avaliacao DATE NOT NULL,
  id_usuario INT NOT NULL,
  id_jogo INT NOT NULL,

  CONSTRAINT fk_avaliacao_usuario
    FOREIGN KEY (id_usuario)
    REFERENCES usuario(id_usuario)
    ON DELETE CASCADE
    ON UPDATE CASCADE,

  CONSTRAINT fk_avaliacao_jogo
    FOREIGN KEY (id_jogo)
    REFERENCES jogo(id_jogo)
    ON DELETE CASCADE
    ON UPDATE CASCADE,

  CONSTRAINT chk_nota
    CHECK (nota >= 0 AND nota <= 10)
);

CREATE TABLE jogo_plataforma (
```

```

id_jogo INT NOT NULL,
id_plataforma INT NOT NULL,

PRIMARY KEY (id_jogo, id_plataforma),

CONSTRAINT fk_jp_jogo
FOREIGN KEY (id_jogo)
REFERENCES jogo(id_jogo)
ON DELETE CASCADE
ON UPDATE CASCADE,

CONSTRAINT fk_jp_plataforma
FOREIGN KEY (id_plataforma)
REFERENCES plataforma(id_plataforma)
ON DELETE CASCADE
ON UPDATE CASCADE

```

};

3.2 Inserção de dados

Depois da criação das tabelas, foram inseridos dados de teste. Esses registros servem para demonstrar as consultas, testar as cardinalidades e visualizar o funcionamento do sistema web com informações reais no banco.

```

INSERT INTO usuario (nome, email, senha) VALUES
('Carlos Silva', 'carlos@email.com', '123456'),
('Mariana Souza', 'mariana@email.com', 'abc123'),
('Joao Pereira', 'joao@email.com', 'senha789');

INSERT INTO genero (nome_genero) VALUES
('Acao'),
('RPG'),
('Esporte');

INSERT INTO desenvolvedora (nome_desenvolvedora, pais) VALUES
('Rockstar Games', 'Estados Unidos'),
('CD Projekt Red', 'Polonia'),
('EA Sports', 'Estados Unidos');

INSERT INTO plataforma (nome_plataforma) VALUES
('PC'),
('PlayStation 5'),
('Xbox Series X');

INSERT INTO jogo (
    titulo,
    ano_lancamento,
    classificacao_indicativa,
    id_genero,
    id_desenvolvedora
) VALUES
('Grand Theft Auto V', 2013, '18', 1, 1),
('The Witcher 3', 2015, '16', 2, 2),
('EA Sports FC 24', 2023, 'Livre', 3, 3);

INSERT INTO jogo_plataforma (id_jogo, id_plataforma) VALUES
(1, 1),
(1, 2),
(1, 3),
(2, 1),
(2, 2),
(3, 2),
(3, 3);

INSERT INTO avaliacao (
    nota,
    comentario,
    data_avaliacao,
    id_usuario,
    id_jogo
) VALUES
(9.5, 'Jogo muito divertido.', '2026-05-10', 1, 1),
(10.0, 'Historia excelente.', '2026-05-11', 2, 2),
(8.0, 'Bom jogo de futebol.', '2026-05-12', 3, 3);

```

3.3 Consultas SQL utilizadas no trabalho

As consultas abaixo foram montadas pensando nas telas do sistema e nos requisitos do projeto. A primeira consulta usa mais de três tabelas e também utiliza JOIN explícito, pois cruza dados de jogo, gênero, desenvolvedora, plataforma e avaliação.

Consulta 1 - Listagem completa dos jogos

Objetivo: Mostra cada jogo com gênero, desenvolvedora, plataformas e média de avaliação. Essa consulta é a base da tela principal de jogos.

```
SELECT
    j.id_jogo,
    j.titulo,
    j.ano_lancamento,
    j.classificacao_indicativa,
    g.nome_genero,
    d.nome_desenvolvedora,
    GROUP_CONCAT(DISTINCT p.nome_plataforma SEPARATOR ', ') AS plataformas,
    AVG(a.nota) AS media_avaliacao
FROM jogo j
INNER JOIN genero g ON j.id_genero = g.id_genero
INNER JOIN desenvolvedora d ON j.id_desenvolvedora = d.id_desenvolvedora
LEFT JOIN jogo_plataforma jp ON j.id_jogo = jp.id_jogo
LEFT JOIN plataforma p ON jp.id_plataforma = p.id_plataforma
LEFT JOIN avaliacao a ON j.id_jogo = a.id_jogo
GROUP BY j.id_jogo, j.titulo, j.ano_lancamento,
j.classificacao_indicativa, g.nome_genero, d.nome_desenvolvedora
ORDER BY j.id_jogo DESC;
```

Jogo	Gênero	Desenvolvedora	Plataformas	Média
EA Sports FC 24	Esporte	EA Sports	PlayStation 5, Xbox Series X	8.0
The Witcher 3	RPG	CD Projekt Red	PC, PlayStation 5	10.0
Grand Theft Auto V	Acao	Rockstar Games	PC, PlayStation 5, Xbox Series X	9.5

Consulta 2 - Relatório consolidado pela VIEW

Objetivo: Busca os dados prontos da view de relatório, facilitando a apresentação de média, quantidade, menor nota e maior nota.

```
SELECT *
FROM vw_relatorio_jogos
ORDER BY titulo;
```

Jogo	Qtd. avaliações	Média	Menor	Maior
EA Sports FC 24	1	8.0	8.0	8.0
Grand Theft Auto V	1	9.5	9.5	9.5
The Witcher 3	1	10.0	10.0	10.0

Consulta 3 - Avaliações com usuário e jogo

Objetivo: Lista quem avaliou, qual jogo foi avaliado, a nota dada e o comentário registrado.

```
SELECT
    a.id_avaliacao,
    u.nome AS usuario,
    j.titulo AS jogo,
    a.nota,
    a.comentario,
    a.data_avaliacao
FROM avaliacao a
INNER JOIN usuario u ON a.id_usuario = u.id_usuario
INNER JOIN jogo j ON a.id_jogo = j.id_jogo
ORDER BY a.id_avaliacao DESC;
```

Usuario	Jogo	Nota	Comentario
Joao Pereira	EA Sports FC 24	8.0	Bom jogo de futebol.
Mariana Souza	The Witcher 3	10.0	Historia excelente.
Carlos Silva	Grand Theft Auto V	9.5	Jogo muito divertido.

Consulta 4 - Quantidade de jogos por plataforma

Objetivo: Conta quantos jogos estão vinculados a cada plataforma cadastrada.

```
SELECT
    p.nome_plataforma,
    COUNT(jp.id_jogo) AS quantidade_jogos
FROM plataforma p
LEFT JOIN jogo_plataforma jp ON p.id_plataforma = jp.id_plataforma
GROUP BY p.id_plataforma, p.nome_plataforma
ORDER BY quantidade_jogos DESC;
```

Plataforma	Quantidade de jogos
PlayStation 5	3
PC	2

3.4 Objeto de banco de dados - VIEW

A opção escolhida foi uma VIEW, pois ela deixa a consulta de relatório mais limpa. Em vez de repetir toda a lógica com JOIN e funções de agregação, o sistema pode consultar uma visão pronta do banco.

```
CREATE VIEW vw_relatorio_jogos AS
SELECT
    j.id_jogo,
    j.titulo,
    g.nome_genero,
    d.nome_desenvolvedora,
    COUNT(a.id_avaliacao) AS quantidade_avaliacoes,
    AVG(a.nota) AS media_avaliacao,
    MIN(a.nota) AS menor_nota,
    MAX(a.nota) AS maior_nota
FROM jogo j
INNER JOIN genero g
    ON j.id_genero = g.id_genero
INNER JOIN desenvolvedora d
    ON j.id_desenvolvedora = d.id_desenvolvedora
LEFT JOIN avaliacao a
    ON j.id_jogo = a.id_jogo
GROUP BY
    j.id_jogo,
    j.titulo,
    g.nome_genero,
    d.nome_desenvolvedora;
```

```
SELECT * FROM vw_relatorio_jogos;
```

Justificativa da VIEW

A view ajuda na apresentação do projeto porque mostra que o banco também pode guardar consultas organizadas. Ela resume informações importantes dos jogos, como quantidade de avaliações, média, menor nota e maior nota.

4. SISTEMA WEB

4.1 Tecnologias e arquitetura

A aplicação web foi feita em Node.js com Express. O Express recebe as requisições das páginas, consulta o banco de dados MySQL usando mysql2 e renderiza as telas com EJS. A parte visual usa HTML, CSS e Bootstrap para deixar os formulários e tabelas mais organizados.

Camada	Tecnologia usada	Papel no projeto
Back-end	Node.js + Express	Controla rotas, requisições e respostas
Banco de dados	MySQL	Armazena jogos, usuários, avaliações e relacionamentos
Conexão	mysql2	Permite executar SQL pelo Node.js
Views	EJS	Monta as páginas usando dados vindos do banco
Front-end	HTML, CSS e Bootstrap	Organiza telas, formulários e tabelas

4.2 Estrutura de pastas

A estrutura do projeto foi mantida simples para facilitar a apresentação e a manutenção. As rotas ficaram concentradas no arquivo jogos.js, a conexão no arquivo db.js e as telas dentro da pasta views.

```

biblioteca-jogos/
├── app.js
├── config/
│   └── db.js
├── public/
│   ├── css/
│   └── style.css
├── routes/
│   └── jogos.js
└── views/
    ├── index.ejs
    ├── jogos.ejs
    ├── novo-jogo.ejs
    ├── editar-jogo.ejs
    ├── avaliar-jogo.ejs
    ├── avaliacoes.ejs
    └── relatorio.ejs

```

4.3 Rotas implementadas

Rota	Função
GET /	Abre a tela inicial do sistema
GET /jogos	Lista os jogos cadastrados com gênero, desenvolvedora, plataformas e média
GET /jogos/novo	Abre o formulário para cadastrar um jogo
POST /jogos/novo	Salva jogo, gênero, desenvolvedora, plataforma e vínculo no banco
GET /jogos/editar/:id	Carrega dados de um jogo para edição
POST /jogos/editar/:id	Atualiza informações do jogo
GET /jogos/avaliar/:id	Abre o formulário de avaliação
POST /jogos/avaliar/:id	Registra usuário, nota e comentário
GET /jogos/excluir/:id	Exclui primeiro os vínculos e avaliações, depois o jogo
GET /relatorio	Mostra relatório com quantidade e média das avaliações
GET /avaliacoes	Lista avaliações registradas

4.4 Trechos principais do código

Arquivo app.js

Esse arquivo configura o servidor Express, define o EJS como ferramenta de visualização e libera a pasta public para arquivos estáticos, como CSS.

```

const express = require('express');
const bodyParser = require('body-parser');
const jogosRoutes =
  require('./routes/jogos');

```

```
const app = express();

app.set('view engine', 'ejs');

app.use(bodyParser.urlencoded({ extended: false }));
app.use(express.static('public'));

app.use('/', jogosRoutes);

app.listen(3000, () => {
  console.log('Servidor rodando em http://localhost:3000');
});
```

Arquivo config/db.js

Esse arquivo faz a conexão com o MySQL. A senha foi trocada por sua_senha no relatório por segurança, mas no projeto local ela deve ser ajustada de acordo com o computador usado.

```
const mysql = require('mysql2');

const connection = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password:
    'sua senha',
  database: 'biblioteca_jogos'
});

connection.connect((erro) => {
  if (erro) {
    console.log('Erro ao conectar ao MySQL:', erro);
    return;
  }

  console.log('Conectado ao MySQL!');
});

module.exports = connection;
```

Rota de listagem de jogos

Essa é uma das partes mais importantes do sistema, porque usa vários JOINS para montar uma listagem completa dos jogos. Ela junta título, ano, classificação, gênero, desenvolvedora, plataformas e média das avaliações.

```
router.get('/jogos', (req, res) => {
  const sql = `
    SELECT
      j.id_jogo,
      j.titulo,
      j.ano_lancamento,
      j.classificacao_indicativa,
      g.nome_genero,
      d.nome_desenvolvedora,
      GROUP_CONCAT(DISTINCT p.nome_plataforma SEPARATOR ', ') AS plataformas, AVG(a.nota)
      AS media_avaliacao
    FROM jogo j
    INNER JOIN genero g ON j.id_genero = g.id_genero
    INNER JOIN desenvolvedora d ON j.id_desenvolvedora = d.id_desenvolvedora LEFT
    JOIN jogo_plataforma jp ON j.id_jogo = jp.id_jogo
    LEFT JOIN plataforma p ON jp.id_plataforma = p.id_plataforma
    LEFT JOIN avaliacao a ON j.id_jogo = a.id_jogo
    GROUP BY
      j.id_jogo,
      j.titulo,
      j.ano_lancamento,
      j.classificacao_indicativa,
      g.nome_genero,
      d.nome_desenvolvedora
    ORDER BY j.id_jogo DESC
  `;

  db.query(sql, (erro, jogos) => {
    if (erro) {
      console.log(erro);
      return res.send('Erro ao listar jogos.');
```

Rota de cadastro de jogo

No cadastro, o sistema não insere apenas o jogo. Ele também verifica ou cria o gênero, cria a desenvolvedora, verifica ou cria a plataforma e depois salva o vínculo na tabela jogo_plataforma.

```
router.post('/jogos/novo', (req, res) => {
  const {
    titulo,
    ano_lancamento,
    classificacao_indicativa,
    nome_genero,
    nome_desenvolvedora,
    nome_plataforma
  } = req.body;

  db.query(
    'INSERT IGNORE INTO genero (nome_genero) VALUES (?)',
    [nome_genero],
    (erroGeneroInsert) => {
      if (erroGeneroInsert) {
        console.log(erroGeneroInsert);
        return res.send('Erro ao cadastrar gênero.');

```

Rota de edição de jogo

Na edição, o sistema atualiza os dados principais e também refaz o vínculo da plataforma. Isso evita que o jogo continue ligado a uma plataforma antiga quando o usuário altera o cadastro.

```
router.post('/jogos/editar/:id', (req, res) => {
  const id = req.params.id;

  const {
    titulo,
    ano_lancamento,
    classificacao_indicativa,
    nome_genero,
    nome_desenvolvedora,
    nome_plataforma
  } = req.body;

  db.query(
    'INSERT IGNORE INTO genero (nome_genero) VALUES (?)',
    [nome_genero],
    (erroGeneroInsert) => {
      if (erroGeneroInsert) {
        console.log(erroGeneroInsert)
        return res.send('Erro ao cadastrar gênero.');
      }
    }

    db.query(
      'SELECT id_genero FROM genero WHERE nome_genero = ?',
      [nome_genero],
      (erroGenero, genero) => {
        if (erroGenero || genero.length === 0) {
          console.log(erroGenero);
          return res.send('Erro ao buscar gênero.');
        }
      }

      db.query(
        'INSERT IGNORE INTO plataforma (nome_plataforma) VALUES (?)',
        [nome_plataforma],
        (erroPlatInsert) => {
          if (erroPlatInsert) {
            console.log(erroPlatInsert);
            return res.send('Erro ao cadastrar plataforma.');
          }
        }

        db.query(
          'SELECT id_plataforma FROM plataforma WHERE nome_plataforma = ?',
          [nome_plataforma],
          (erroPlat, plataforma) => {
            if (erroPlat || plataforma.length === 0) {
              console.log(erroPlat);
              return res.send('Erro ao buscar plataforma.');
            }
          }

          db.query(
            'SELECT id_desenvolvedora FROM jogo WHERE id_jogo = ?',
            [id],
            (erroBusca, jogoAtual) => {
              if (erroBusca || jogoAtual.length === 0) {
                console.log(erroBusca);
                return res.send('Erro ao buscar jogo.');
              }
            }

            db.query(
              'UPDATE desenvolvedora SET nome_desenvolvedora = ?
              WHERE id_desenvolvedora = ?',
              [nome_desenvolvedora,
               jogoAtual[0].id_desenvolvedora],
              (erroDev) => {
                if (erroDev) {
                  console.log(erroDev)
                  return res.send('Erro ao editar empresa.');
                }
              }

              const sqlJogo = `
                UPDATE jogo
                SET titulo = ?, ano_lancamento = ?,
                  classificacao_indicativa = ?, id_genero
                WHERE id_jogo = ?
              `;

              db.query(
                sqlJogo,
                [
                  titulo,
                  ano_lancamento,
                  classificacao_indicativa,
                  genero[0].id_genero,
                  id
                ],
                (erroJogo) => {
                  if (erroJogo) {
                    console.log(erroJogo);
                    return res.send('Erro ao editar
                    jogo.');
                  }
                }

                db.query(
                  'DELETE FROM jogo_plataforma
                  WHERE id_jogo = ?',
                  [id],
                  (erroDel) => {
                    if (erroDel) {
                      console.log(erroDel);
                      return res.send('Erro ao
                      atualizar plataforma.');
                    }
                  }

                  db.query(
                    'INSERT INTO
                    jogo_plataforma
                    (id_jogo,
                     id_plataforma) VALUES
                    (id, (?, ?))',
                    [id, (?, ?)]
```

```

        plataforma[id_plataforma],
        (erroJP) => {
            if (erroJP) {
                console.log(erroJP);
                return
                res.send('Erro
                ao salvar
                plataforma.');
```

Rota de avaliação

A avaliação permite registrar uma nota e um comentário para o jogo. O sistema cria um usuário para a avaliação e salva a data atual usando CURDATE().

```

router.post('/jogos/avaliar/:id', (req, res) => { const
    id_jogo = req.params.id;

    const {
        nome_usuario,
        nota,
        comentario
    } = req.body;

    const emailAutomatico =
        nome_usuario.toLowerCase().replace(/s+/g, '') +
        Date.now() +
        '@avaliacao.com';

    db.query(
        'INSERT INTO usuario (nome, email, senha) VALUES (?, ?, ?)',
        [nome_usuario, emailAutomatico, '123'],
        (erroUsuario, usuarioResultado) => {
            if (erroUsuario) {
                console.log(erroUsuario);
                return res.send('Erro ao cadastrar usuário.');
```

Rota de exclusão

A exclusão foi feita em etapas. Primeiro o sistema apaga os vínculos com plataformas e as avaliações relacionadas ao jogo. Depois disso, o jogo é excluído. Essa ordem evita erro de chave estrangeira.

```

router.get('/jogos/excluir/:id', (req, res) => {
    const id = req.params.id;

    db.query(
        'DELETE FROM jogo_plataforma WHERE id_jogo = ?', [id],
        (erroJP) => {
```

```

    if (erroJP) {
      console.log(erroJP);
      return res.send('Erro ao excluir plataformas.');
```

```

    }

    db.query(
      'DELETE FROM avaliacao WHERE id_jogo = ?',
      [id],
      (erroAvaliacao) => {
        if (erroAvaliacao) {
          console.log(erroAvaliacao);
          return res.send('Erro ao excluir avaliações.');
```

```

        }

        db.query(
          'DELETE FROM jogo WHERE id_jogo = ?', [id],
          (erroJogo) => {
            if (erroJogo) {
              console.log(erroJogo);
              return res.send('Erro ao excluir jogo.');
```

```

            }

            res.redirect('/');
          }
        )
      }
    )
  }
}

```

Rota de relatório

O relatório calcula quantidade de avaliações, média, menor nota e maior nota para cada jogo. Ele é útil para mostrar o resultado do sistema funcionando com dados reais.

```

router.get('/relatorio', (req, res) => { const
  sql = `
    SELECT
      j.id_jogo,
      j.titulo,
      g.nome_genero,
      d.nome_desenvolvedora,
      COUNT(a.id_avaliacao) AS quantidade_avaliacoes, AVG(a.nota)
      AS media_avaliacao,
      MIN(a.nota) AS menor_nota,
      MAX(a.nota) AS maior_nota
    FROM jogo j
    INNER JOIN genero g ON j.id_genero = g.id_genero
    INNER JOIN desenvolvedora d ON j.id_desenvolvedora = d.id_desenvolvedora LEFT
    JOIN avaliacao a ON j.id_jogo = a.id_jogo
    GROUP BY
      j.id_jogo,
      j.titulo,
      g.nome_genero,
      d.nome_desenvolvedora
    ORDER BY j.titulo
  `;

  db.query(sql, (erro, jogos) => {
    if (erro) {
      console.log(erro);
      return res.send('Erro ao carregar relatório.');
```

```

    }

    res.render('relatorio', { jogos });
  })
}

```

4.5 Telas do sistema

As telas em EJS foram organizadas para cobrir as funções principais do CRUD e também as funções extras de avaliação e relatório.

- index.ejs - tela inicial com atalhos para as principais funções.
- jogos.ejs - listagem dos jogos cadastrados e botões de ação.
- novo-jogo.ejs - formulário de cadastro de jogo.
- editar-jogo.ejs - formulário para alterar dados existentes.
- avaliar-jogo.ejs - tela para inserir nota e comentário.
- avaliacoes.ejs - lista as avaliações já feitas.
- relatorio.ejs - mostra médias e quantidade de avaliações por jogo.

5. EXECUÇÃO, TESTES E FECHAMENTO

5.1 Como executar o projeto

Para rodar o sistema, primeiro é necessário importar o script SQL no MySQL. Depois, a senha do banco deve ser conferida no arquivo config/db.js. Com as dependências instaladas, o servidor pode ser iniciado pelo Node.js.

```
# 1. Importar o arquivo SQL no MySQL
ArquivoSQL.sql

# 2. Entrar na pasta da
aplicação cd biblioteca-jogos

# 3. Instalar as dependências, caso ainda não estejam
instaladas npm install

# 4. Iniciar o servidor
node app.js

# 5. Abrir o sistema no
navegador http://localhost:3000
```

5.2 Pontos testados

- Cadastro de novos jogos com gênero, desenvolvedora e plataforma.
- Listagem dos jogos puxando dados reais do MySQL.
- Edição das informações cadastradas.
- Exclusão respeitando as chaves estrangeiras.
- Registro de avaliação com nota e comentário.
- Relatório com média, menor nota, maior nota e quantidade de avaliações.

Observação do desenvolvimento

A parte mais delicada foi manter a coerência entre o banco e o sistema web, principalmente no relacionamento entre jogo e plataforma. Como existe uma tabela associativa, o cadastro e a edição precisam salvar também esse vínculo, não apenas os dados da tabela jogo.

5.3 Considerações finais

O trabalho conseguiu integrar a modelagem do banco de dados com uma aplicação web funcional. O banco foi estruturado com tabelas separadas, chaves primárias, chaves estrangeiras, restrições e uma view de relatório. Já o sistema web utiliza essas tabelas em telas de cadastro, listagem, edição, exclusão, avaliação e relatório.

A Biblioteca de Jogos ficou como um exemplo prático de aplicação CRUD conectada a um banco relacional. Além disso, o projeto mostra como os relacionamentos pensados no MER e no DER aparecem de verdade no código, principalmente nas consultas com JOIN e na tabela jogo_plataforma.